

CSCE 629 ANALYSIS OF ALGORITHMS · MODULE 07 · LECTURE NOTES

Shortest Paths

Weights change everything, and negative weights change it again.

MODULE OUTCOMES

Explain why BFS fails on weighted graphs.

Implement Dijkstra and justify its correctness by the greedy argument.

Explain why Dijkstra fails on negative edges and what Bellman–Ford does instead.

Use Bellman–Ford to detect negative cycles.

Choose the right algorithm from the graph's properties.

1 Relaxation: the one operation

Every algorithm in this module maintains an array $d[]$ of distance *estimates*, each an upper bound on the true shortest distance, and improves them with a single operation:

$$\text{if } d[u] + w(u,v) < d[v] \text{ then } d[v] \leftarrow d[u] + w(u,v)$$

Estimates start at ∞ (except $d[s] = 0$), decrease monotonically, and never drop below the truth. Once every edge satisfies $d[v] \leq d[u] + w(u,v)$, no further improvement is possible and the estimates are exact.

THE ONLY DIFFERENCE BETWEEN THESE ALGORITHMS IS THE ORDER

Dijkstra relaxes edges out of the closest unfinished vertex, which guarantees each vertex is finalised once.

Bellman–Ford relaxes every edge repeatedly, $V-1$ times, making no assumption about order at all.

DAG relaxation relaxes edges in topological order, which guarantees a vertex is finalised before any of its successors are touched.

Seeing them as three relaxation schedules rather than three algorithms makes their complexities and their limitations follow naturally.

2 Why BFS is not enough

BFS explores strictly in order of edge count, so it finds the path with the fewest *edges*. Once weights exist, that is a different problem: a two-edge path of total weight 3 is shorter than a one-edge path of weight 100, and BFS commits to the latter.

The repair is to explore in order of accumulated *distance* rather than edge count — that is, to replace the FIFO queue with a priority queue keyed by $d[v]$. The result is Dijkstra's algorithm, and the structural similarity to BFS is not a coincidence; it is the same traversal with a different container, exactly as in Module 06.

3 Dijkstra

```
import heapq

def dijkstra(G, s):
    d = {v: float('inf') for v in G}
    d[s] = 0
    pq = [(0, s)]
    done = set()

    while pq:
        du, u = heapq.heappop(pq)
        if u in done:           # stale entry from an earlier, worse estimate
            continue
        done.add(u)           # d[u] is final from here on

        for v, w in G[u]:
            if du + w < d[v]:
                d[v] = du + w
                heapq.heappush(pq, (d[v], v))

    return d
```

LAZY DELETION INSTEAD OF DECREASE-KEY

Textbook Dijkstra calls decrease-key to update a vertex's priority in place, keeping the heap at $O(V)$ entries.

Real implementations push a duplicate entry and discard stale pops, as above. The heap grows to $O(E)$ entries, which costs a constant factor in the log, and in exchange you avoid maintaining handles into the heap — simpler code that is usually faster.

This is worth knowing because the gap between the textbook presentation and the deployed one is wide here, and the textbook version is the one people waste time implementing.

3.1 Correctness, and where the assumption enters

Claim. When a vertex u is popped from the priority queue, $d[u]$ equals the true shortest distance from s to u .

Proof sketch. Suppose not, and let u be the first vertex popped with $d[u]$ greater than the true distance. Then there is a genuinely shorter path P from s to u . Since u is being popped now, P must leave the finished set at some point; let x be the first unfinished vertex on P . The portion of P up to x has length at most the length of all of P , which is less than $d[u]$. But then $d[x] < d[u]$, so x would have been popped before u .

Contradiction.

FIND THE ASSUMPTION

The step 'the portion of P up to x has length at most the length of all of P ' is where non-negativity is used, and it is the only place.

With a negative edge later in P , the full path can be *shorter* than its own prefix. The inequality reverses and the proof fails.

This is why Dijkstra is wrong on negative edges — not as an arbitrary restriction but as a direct consequence of its greedy argument. A greedy algorithm is only as good as its exchange argument, which is Module 09's theme.

Priority queue	Complexity	When
Binary heap	$O((V + E) \log V)$	The default. Nearly always right.
Fibonacci heap	$O(E + V \log V)$	Asymptotically better, constants large enough that it rarely wins. The canonical example of theory outrunning practice.
Simple array	$O(V^2)$	Better on dense graphs where $E \approx V^2$, since the log disappears.
Bucket queue	$O(E + VC)$	Dial's algorithm, for small integer weights bounded by C .

4 Bellman–Ford

Make no assumption about order: simply relax every edge in the graph, and repeat.

```
def bellman_ford(V, edges, s):
    d = {v: float('inf') for v in V}
    d[s] = 0

    for _ in range(len(V) - 1):          # V-1 rounds
        for (u, v, w) in edges:
            if d[u] + w < d[v]:
                d[v] = d[u] + w

    for (u, v, w) in edges:             # one more round
        if d[u] + w < d[v]:
            raise NegativeCycle(v)      # still improving -> neg. cycle
    return d
```

4.1 Why V–1 rounds

A shortest path visits no vertex twice — revisiting a vertex means traversing a cycle, and with no negative cycles that cycle has weight ≥ 0 , so removing it does not lengthen the path. A simple path therefore has at most V–1 edges.

By induction, after round k every shortest path using at most k edges has been found: round k relaxes the k-th edge of such a path after rounds 1..k–1 established the prefix. So V–1 rounds suffice, giving $O(VE)$.

4.2 Negative cycles

THE ANSWER IS $-\infty$, WHICH IS WHY DETECTION MATTERS

If a negative-weight cycle lies on some path from s to t, you can traverse the cycle repeatedly and reduce the total without limit. The shortest distance is $-\infty$, and 'shortest path' is not merely hard to compute — it is undefined.

After V–1 rounds everything should be settled. If a V-th round still improves some estimate, a negative cycle is reachable, and reporting it is the correct output.

Arbitrage detection is exactly this: take $-\log$ of each exchange rate, and a sequence of trades that multiplies to more than 1 becomes a cycle whose weights sum to less than 0. Bellman–Ford finds it.

5 The other cases

5.1 DAGs

On a directed acyclic graph, relax edges in topological order. Each vertex is finalised before any vertex it points to is examined, so one pass suffices: $O(V + E)$, and negative weights are no obstacle because

nothing is ever revisited. Worth remembering — scheduling and dependency problems are frequently DAGs, and this is both faster and simpler than Bellman–Ford.

5.2 A*

Dijkstra expands outward in every direction because it has no information about where the target is. A* adds a heuristic $h(v)$ estimating the remaining distance and orders the queue by $d[v] + h(v)$ — preferring vertices that look like progress toward the goal.

- Correct if h is **admissible**: it never overestimates the true remaining distance.
- If additionally h is **consistent** ($h(u) \leq w(u,v) + h(v)$), no vertex is expanded twice and the implementation is as simple as Dijkstra's.
- $h = 0$ is admissible and recovers Dijkstra exactly. Euclidean distance is admissible on a road network or a game map.
- The better the heuristic, the fewer vertices expanded — routinely an order of magnitude on a grid map, which is why A* is the standard pathfinder in games and robotics.

5.3 All pairs

Floyd–Warshall computes shortest paths between every pair in $O(V^3)$ and three lines:

```
for k in V:
    for i in V:
        for j in V:
            d[i][j] = min(d[i][j], d[i][k] + d[k][j])
```

THIS IS A DYNAMIC PROGRAM, AND A USEFUL PREVIEW OF MODULE 10

The invariant: after the iteration for k , $d[i][j]$ is the shortest path from i to j using only vertices $\{1..k\}$ as intermediates.

The subproblem is therefore 'shortest i – j path restricted to the first k intermediates', and each is solved from smaller ones — exactly the shape of a dynamic program.

This also explains why k **must** be the outermost loop. Putting it inside gives a subtly wrong answer, and the reason is a dependency-order violation: the DP table would be read before the entries it depends on were written. Module 10 makes this general.

6 Choosing

Graph	Algorithm	Complexity	Note
Unweighted	BFS	$O(V + E)$	Do not reach for Dijkstra here.
Non-negative weights	Dijkstra	$O((V + E) \log V)$	The default.

Graph	Algorithm	Complexity	Note
Any weights, no negative cycles	Bellman–Ford	$O(VE)$	Also detects negative cycles.
DAG, any weights	Topological relaxation	$O(V + E)$	Faster and simpler than Bellman–Ford. Check for acyclicity first.
Known target, good heuristic	A*	Varies	Often an order of magnitude better than Dijkstra.
All pairs, dense	Floyd–Warshall	$O(V^3)$	Three lines; handles negative edges.
All pairs, sparse, negative edges	Johnson	$O(VE + V^2 \log V)$	Reweight with Bellman–Ford, then run Dijkstra from each vertex.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.006 — Dijkstra, Bellman-Ford, DAG shortest paths	Three lectures covering this module, with the relaxation framing made explicit.
Jeff Erickson, Algorithms — Shortest Paths chapter	Presents all of them as one generic relaxation algorithm with different orderings. The best treatment of this framing anywhere.
Amit Patel — Introduction to A* (Red Blob Games)	Interactive, and the clearest explanation of admissibility and heuristic quality available. Directly useful for game work.
CSES Problem Set — Shortest Path section	Auto-graded problems covering every algorithm here.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement Dijkstra with lazy deletion. Verify on a graph where the fewest-edges path differs from the shortest-weight path, and confirm BFS gets it wrong.
- 2 Construct a small graph with one negative edge where Dijkstra returns an incorrect distance. Trace the execution and identify the exact step where the correctness proof fails.
- 3 Implement Bellman–Ford with negative-cycle detection. Build a currency-exchange graph with $-\log$ rates and detect an arbitrage opportunity.

- 4 Implement DAG shortest paths by topological relaxation. Verify it handles negative weights correctly and measure it against Bellman–Ford on the same DAG.
- 5 Implement A* on a grid with obstacles. Compare the number of expanded vertices against Dijkstra for the same start and goal, with Euclidean and Manhattan heuristics.
- 6 Make your A* heuristic inadmissible by scaling it up by $1.5\times$. Show it expands fewer nodes and sometimes returns a suboptimal path. Quantify the trade.
- 7 Implement Floyd–Warshall. Then move the k loop innermost and find an input where the answer becomes wrong. Explain it as a dependency-order violation.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What single operation underlies every algorithm in this module, and what distinguishes the algorithms?
- 2 Why does BFS fail on weighted graphs, and what minimal change fixes it?
- 3 Reproduce the step in Dijkstra's correctness proof that requires non-negative weights.
- 4 Why exactly $V-1$ rounds in Bellman–Ford? What does a V -th improving round mean?
- 5 Why is the shortest path undefined in the presence of a reachable negative cycle?
- 6 What makes an A* heuristic admissible, and what happens to the result if it is not?
- 7 Why must k be the outermost loop in Floyd–Warshall?